

Computational Neuroscience

AY 2023-24

Laboratory Assignment 1 (LAB1)

Bonus Track Assignment#2: Implementing Liquid State Machines (LSMs)

General Information

The subfolder for this lab should be called “LAB1” and should include the scripts and the other files as requested in the assignment below. You can organize the code as you wish, implementing all the helper functions that you need, provided that these are included in the subfolder and are appropriately called in the scripts.

Bonus track assignments are meant to be for those who finish early, but they are not formally required for completing the Lab Assignment.

Detailed instructions for submissions are given in the lecture note “Intro to the course”.

The recommended programming language is Python (e.g., with NumPy, or TensorFlow), but you are allowed to use MATLAB.

If you use Python, you will provide:

- the source files in one of these forms:
 - a collection of the scripts (.py files) used to solve the assignment, with a main script that calls all the necessary other scripts;
 - a Jupyter notebook (.ipynb file) with all the necessary code.
- the output data as a collection of pickle or json files, or in a single pickle/json file
 - in this case it is required to include a script to load the saved data in the chosen format.

If you use MATLAB, you will provide:

- the source files as a collection of the scripts (.m files) used to solve the assignment, or as a live script (.mlx file) with all the necessary code.
- the output data as a collection of binary .mat files, or in a single .mat file.

In both cases, the required figures can be provided in .pdf files or in a graphics format (e.g., png, jpg). When multiple images are required, you are allowed to generate a single file including multiple subplots (or pages, if appropriate).

Supporting Material

Papers on the Izhikevich's Model

[1] E.M. Izhikevich, "Simple model of spiking neurons." IEEE Transactions on neural networks 14.6 (2003): 1569-1572.

Available online at: <http://izhikevich.org/publications/spikes.pdf>

[2] E.M. Izhikevich, "Which model to use for cortical spiking neurons?." IEEE transactions on neural networks 15.5 (2004): 1063-1070.

Available online at: <http://izhikevich.org/publications/whichmod.pdf>

Web page: <http://izhikevich.org/publications/whichmod.htm>

Starting point for your code

The Liquid of the LSM should be implemented by using the pattern of connectivity and the parameters (at least initially), following a Python version of the code originally provided by Izhikevich, can be found in this notebook

<https://drive.google.com/file/d/1-PRKYAVRpNezoSJQQd-ZRRhUu1QqNeMu/view?usp=sharing>

You will need to vary several hyper-parameters, e.g., the number of excitatory and inhibitory neurons and the strength of the fixed liquid's weights.

The MATLAB code by Izhikevich, which can be found at

<http://www.izhikevich.org/publications/net.m>

and a variant to it that you can find at

<https://drive.google.com/file/d/1GFb2UY97McTy9z0kMI-S8DYwatRYaT2z/view?usp=sharing>

Python documentation

<https://www.learnpython.org/>

<https://numpy.org/>

<https://matplotlib.org/>

Matlab documentation

Matlab User's Guide <https://www.mathworks.com/help/index.html>

Matlab documentation using the *help* command

Bonus Track Assignment: Solving the Sunspot task

The Sunspot task consists in a next-step prediction (autoregressive, a particular case of transduction) on a time-series consisting in monthly averaged solar sunspots.

The dataset can be found in .csv format the following link:

<https://drive.google.com/file/d/1GK5fqzuAGoo466PlxhnwxtSP0r3uDFWa/view?usp=sharing>

The dataset can also be imported from MATLAB, using the command `load solar_dataset`. In this latter case, you will need to convert the data format from cell to mat using the command `cell2mat`.

Split the data into input and target signals. As this is an autoregressive task you will need to refer to the same time-series. For example, in Numpy, if the solar data is in the variable **whole_dataset**, you can set **input_dataset = whole_dataset[:, :-1]** and **target_dataset = whole_dataset[:, 1:]**. In MATLAB, you can set **input_dataset = whole_dataset(:, 1:end-1)**, and **target_dataset = whole_dataset(:, 2:end)**.

Properly separate input and target data, then split the available data in

- training (first 2398 time steps), and
- test set (the subsequent 500 time steps).

For model selection you will use the data in the training set, with a further split in training and validation. Try to plot the time series data using the command `plot`.

Solve this regression task with an LSM, where:

- The liquid is given by a layer of interconnected Izhikevich neurons, where the input is added as external applied current
- The readout is a single neuron, trained by pseudo-inversion. Suppose that the liquid states for the training samples are stored in the variables `trainStates`, and the corresponding target output values are in the variable `trainTargets`, then the readout weight matrix can be computed in one go as:

```
Wout = trainTargets @ np.linalg.pinv(trainStates) # in Numpy
```

```
Wout = trainTargets * pinv(trainStates) % in MATLAB
```

- Now it is possible to compute the output of the LSM as, for example:

```
trainOutput = Wout @ trainStates # in Numpy
```

```
trainOutput = Wout * trainStates % in MATLAB
```

and analogously for the test time-steps.

Use a hold-out model selection scheme to choose a suitable number of excitable and inhibitory neurons in the liquid (and other hyper-parameters such as the scalings of the input and recurrent connections).

The output of this bonus track assignment should then consist of the following data, pertaining only to the selected hyper-parametrization:

- The source code
- Training, validation and test errors. Use the Mean Absolute Error (MAE).
- Target vs output plot (i.e., 2 lines, one for the target and one for the output for increasing time). Create one such plot for the training data, and one for the test data.